Highlights

Educational Models for Cognition: Methodology of Modeling Intellectual Skills for Intelligent Tutoring Systems

Oleg Sychev

- Thought Process Trees is a new language of modeling reasoning for learning
- Recommendations for effective building of trees for verifying learners' answers
- Thought Process Trees can be formalized and executed
- The proposed language significantly reduces time of developing intelligent tutors
- It lets classify problems, define possible errors, and generate educational dialogue

Educational Models for Cognition: Methodology of Modeling Intellectual Skills for Intelligent Tutoring Systems

Oleg Sychev[a]

[a]*Software Engineering Department, Lenin Ave, 28, Volgograd, 400005, Russian Federation*

Abstract

Automation of teaching people new skills requires modeling of human reasoning because human cognition involves active reasoning over the new subject domain to acquire skills that will later become automatic. The article presents Thought Process Trees — a language for modeling human reasoning that was created to facilitate the development of intelligent tutoring systems, which can perform the same reasoning that is expected of a student and find deficiencies in their line of thinking, providing explanatory messages and allowing them to learn from performance errors. The methodology of building trees which better reflect human learning is discussed, with examples of design choices during the modeling process and their consequences. The characteristics of educational modeling that impact building subject-domain models for intelligent tutoring systems are discussed. The trees were formalized and served as a basis for developing a framework for constructing intelligent tutoring systems. This significantly lowered the time required to build and debug a constraint-based subject-domain model. The framework has already been used to develop five intelligent tutoring systems and their prototypes and is being used to develop more of them.

Keywords: reasoning modeling, constraint-based modeling, intelligent tutoring systems

1. Introduction

Modern education, preparing people to live in a technologically advanced, digital society, includes developing a wide range of hard and soft skills. There

is a great demand in labour market for engineers and IT-specialists, whose hard skills are heavily based on skills developed in cognitive domain, and their jobs often require regular on-the-job learning [1]. According to widely-used Bloom's taxonomy of educational objectives, skills in the cognitive domain are based on the levels of knowledge (remembering facts), comprehension (which includes applying subject-domain rules in standard situations) and applications (which includes selecting the applicable subject-domain rules according to the situation and making connections between them) [2]. All those tasks often require strictly following specified mental procedures. Failure to learn precise thinking procedures significantly contributes to high failure rates and lowers self-efficacy for students. A common problem in many STEM courses is the lack of teacher time: often, students can only solve a few (or just one) learning problems per topic and receive feedback on the quality of their solutions and the errors they made. According to Ohlsson's theory of learning [3], correcting performance errors plays an important part in learning, allowing learners to refine their mental models, which at first tend to be too general, and specializing knowledge structures so that they will be applied in their respective contexts. Students who do not receive enough feedback on their errors to correct their mental models and master the necessary competencies can fail the course. Large class size causes shortage of teacher time and increases the number of failing students [4]. Manual correction and grading of many similar assignments is time- and labor-intensive [5].

Some of those problems can be solved by automating verifying students' work and providing computer-generated feedback. Educational feedback is the information provided to the student regarding to their effort at learning. E-learning and online learning have been developing significantly recently, boosted by lockdowns caused by the COVID-19 outbreak; they now compete with traditional in-class learning and successfully complement it in hybrid learning environments. Automatic tutors do not have time restrictions of human personnel. Their chief limitations are the number of possible training tasks on the topic and the kind of feedback they provide. Almost every automatic tutor provides some feedback, but the amount of information in them differs significantly. Glover and Brown categorize feedback by what they call "depth of feedback explanation" into 3 categories [6]:

1. acknowledging issue without correcting advice,
2. providing a correct response along with the acknowledged issue,

3. providing the reason why the student's answer is wrong or the expected answer is correct.

Those categories of feedback were formulated by analyzing human-provided written feedback. Showing only that the error was made does very little to allow learning. When implemented in automatic tutoring and testing systems, that sort of feedback results in students asking many questions to teachers during office hours or in online forums because students have no other way to learn what was wrong and how to avoid those errors in the future. Providing a complete correct response and a teacher-written explanation of why that response is correct reduces the number of questions insignificantly. However, providing parts of correct responses provokes interest and significantly decreases the number of questions asked to teachers [7]. Providing the reason why the student's answer is wrong is even more effective, but it can only be automated for closed-answer questions for which the set of possible answers is limited and predictable.

Automatic generation of educational feedback requires more formalized and detailed classification. In particular, during the development of different kinds of automatic tutors with open and closed answers, the following kinds of feedback were used:

- correctness of response — simple feedback, showing whether the answer is correct or not;
- error place — highlighting the particular place in the student's answer or solution that is wrong;
- kinds of errors — naming the type of errors that were made;
- full correct answer (worked example) — providing a correct answer or solution after an error is made;
- partial correct answer (hint) — providing a part of a correct answer or solution solving the immediate problem, but letting the student continue solving the task;
- explaining why the demonstrated answer is correct — often referencing previously taught material;
- providing the explanation of why the student's answer is wrong.

When the goal is to let the student learn without asking human teachers, the most important kinds of feedback are showing places of errors, kinds of errors, partially correct answers, and, if possible, explaining why the answer is wrong. The last kind of feedback significantly speeds up learning because it specifically provides students with the information necessary to set correct contexts for the pieces of knowledge the student tries to apply. Showing places of errors often causes guesswork on how to improve them, and showing correct answers can make students guess why they are correct in the given circumstances. Explanatory feedback is more powerful whenever possible.

This article describes the method of modeling intellectual skills and lessons in subject-domain modeling learned during the development of several intelligent tutoring systems (ITS). The chief goal of the article is discussing the lessons learned during modeling of different kinds of tasks and provide guidance on building effective subject-domain models based on several years of supervision of development of such models in the research group on modern technologies of education in Volgograd State Technical University because the experience shows that the quality of the subject-domain model is an important factor in determining the kinds of automatically provided feedback and so the success of the automatic tutor.

Nowadays, machine learning techniques and large language models, in particular, receive a lot of attention. They allowed for important results in many fields. In ITS, they were successfully used to generate motivational feedback [8], help in self-directed learning [9], preliminary analysis of students' essays [10], and generating personalized learning paths [11]. However, using them in domain modeling is more problematic because of their tendency to make errors now and then, which is especially problematic in STEM disciplines that require precision in learning. The wrong reaction of the ITS to a student's error can seriously harm either the student's learning or the trustworthiness of the system for the student. For example, the accuracy of a GPT-4-based model in rough labeling of errors in the programming tasks was less than 70% in such important categories as "computation", "branching", and "conceptual" even when the input was augmented with descriptions [12]. Other researchers also conclude that the accuracy of rough error causes identification by machine learning methods is too low for directly providing feedback to the learner and can only be used to lower the time of providing feedback by instructors [13]. A rule-based approach is still used in providing personalized feedback in STEM domains like programming [14]. That means that efficiently providing automatic feedback requires developing hybrid AI

tutoring systems [15] with significant symbolic AI components. This article is devoted to developing symbolic AI techniques for intelligent tutoring systems.

For clarity, the rest of the article will use the following conventions. Human teachers and tutors will be named "teachers." They will also be named "authors" if they are involved in developing educational models or learning problems that are used in automatic tutors. The terms "tutor" and "tutoring system" will be used for computer programs that are automatic tutors. Intelligent tutoring systems are a kind of automatic tutors; intelligent tutoring systems analyze the students' responses and provide individualized feedback and (or) select learning problems to solve according to the student's behaviour [16]. Intelligent tutoring systems can use two kinds of artificial intelligence: machine-learning techniques and symbolic artificial intelligence for modeling. Subject-domain modeling is particularly important for ITS because their models of students' knowledge are built on top of subject-domain models, so the quality of subject-domain modeling directly influences the overall quality of the tutoring system.

Section 2 reviews the related works. Section 3 describes the graphical notation of Thought Process Trees used in the rest of the article. The crucial section 4 shows the methodology of modeling intellectual skills using Thought Process Trees for verifying students' solutions to learning problems of different natures. Section 5 describes a way to formalize Thought Process Trees to serve as a basis of a framework for developing intelligent tutoring systems. Section 6 discusses the usage of different kinds of Thought Process Trees in automatic tutors and tutoring systems.

2. Related work

2.1. Subject-domain modeling

Subject-domain modeling has been an important part of developing intelligent tutoring systems for a long time [16]. Usually, they consist of bits of knowledge (items, topics, rules, and so on) that the learner must acquire and can track relationships between them. However, researchers had different views on the functions and desired properties of formal subject-domain models. They can be divided into two big groups: structural and content-related models.

- *Structural models* capture only the structure of the subject domain. The actual knowledge taught to ITS users is not a part of the model;

it is stored separately (for example, in associated learning materials, natural-language definitions and so on).

- *Content-related models* strive to capture the studied content in the formal form as a part of the model.

2.1.1. Structural domain models

are more widely used. They can contain relationships like prerequisite knowledge. They can be designed by experts or learned from data. The popular modeling paradigms for structural domain models include probabilistic models like Bayesian networks (direct acyclic graphs of conditional links between variables) [17, 18, 19] and models created by machine-learning techniques. Machine learning can be used to model typologies of skills [20], automate hints generation [21], cluster erroneous solutions to find misconceptions [22], and refine skill models [23]. Ontological models can also be used to capture the structure of the subject domain separately from procedural knowledge [24] to create purely structural models, even though methods of ontological modeling can be used for creating content-related models. While structural models are relatively easy to create, they have limited usage in intelligent tutoring systems because they capture the knowledge structure and, sometimes, some declarative knowledge — but they do not contain procedural knowledge, which lies at the heart of skills developed using ITS.

2.1.2. Content-related domain models

try to formalize the procedural knowledge that is taught to students. Models of this kind can be used to solve and classify learning problems, generate hints and worked examples, find violations of subject-domain rules, and solve other tasks. They can also contain prerequisite knowledge relationships, but unlike structural models, content-related models can infer this kind of relationship from other model data. There are several approaches to content-related domain modeling. Behaviour graphs [25] capture the problem-solving process for particular learning problems, allowing building automatic tutors based on example tracing. They work well for domains with small solution spaces but do not scale well to domains with many possible ways to solve the problem. Two chief ways of modeling domain content is rule-based models (for example, [26, 27]), which strive to capture the problem solving by using production rules and constraint-based models (e.g., [28]), which verify solutions using sets of constraints to determine violations of subject-domain rules.

The major difference is that rule-based models capture the problem-solving process, and so excel at generating hints and showing worked examples. In contrast, constraint-based models capture what can go wrong during problem solving and excel at generating explanations of students' errors.

2.2. Scaffolding

Another important dimension of providing feedback is scaffolding, i.e., providing feedback in tiny, measured steps instead of a big blob of feedback for the entire student's solution [29]. Scaffolding eases learning by breaking material into small, digestible chunks. Scaffolding can be classified according to two dimensions: scope and method. Two major kinds of scope can be identified:

1. *problem-level scaffolding* means breaking solving a big problem into smaller steps which can be analyzed separately;
2. *step-level scaffolding* is used to break complex explanatory texts into smaller chunks, which is useful for struggling students.

Researchers used several methods of pedagogical scaffolding:

1. *checking the correctness* (applicable only at problem level) — is usually done by verifying answers to each problem-solving step separately and showing which ones are incorrect (e.g., see [30]);
2. *providing information* — giving specialized hints and advice regarding the problematic solution step or part of explanation [31];
3. *asking questions* — scaffolding by asking small, pointed questions about facts and subject-domain rules necessary to solve the problem at hand or understand what is wrong[32].

While scaffolding by checking the correctness of each step is adopted by many researchers and teachers because it is easy to implement, it gives mixed results in affecting students learning [33, 34, 35]. Providing information on solving problematic steps is often useful as people are susceptible to new information after failure, but it makes the learner a passive recipient of information. On the contrary, asking questions stimulates brain activity, thinking and information retrieval and can lead to "Eureka!" moments when the learner realizes where they were wrong. Asking questions is mostly useful in step-level scaffolding. Nicol argues for improving dialogue qualities of learning feedback [36], but he does not go further than gathering students' impressions and using peer dialogues in addition to the teacher's feedback.

Most of the research in scaffolding includes human-authored scaffolding hints and questions, which should be defined for each learning problem or type of learning problem. That makes the development of automatic intelligent tutors very labour intensive, limiting their development and usage. A promising approach to facilitate the development of automatic cognitive intelligent tutors is developing problem-independent languages and tools for modeling the taught skills to lower the workload of developing subject-domain models of intelligent tutoring systems. Out of existing research in automatizing scaffolding, to my best knowledge, only one work [31] dedicated to scaffolding based on the model of problem-solving activity in the formal form. O'Rourke et al. developed Thought Process Language (TPL), similar to structural programming languages; it allows one to write problem-solving algorithms and generate hints in two forms: textual explanations derived from natural-language templates for statements, operators and variable — and interface highlights. Thought Process Language does not model constraints defining correct solutions, so it is not suited well to model tasks with many alternative solution paths and cannot be used for active, question-based scaffolding aimed at understanding what is wrong in the student's solution.

3. Thought Process Trees

The language used to model the subject domain should be readable for machine and human experts who will model their subject domains; readability by learners is a bonus as it adds a new way to transfer the skill. One of the knowledge models with the best combination of human and machine readability is Decision Trees, which has been used in knowledge modeling and machine learning since the XX century [37]. They can be presented in both graphic and textual forms. However, decision trees cannot fully model reasoning because of their limited capabilities: they have only decision nodes, outcome nodes and chance nodes [38]. When describing reasoning during problem solving, there is a need to use additional kinds of actions: looking for an object satisfying specific criteria, calculating, checking several independent branches, and iterating over a set of objects. By reasoning we understand performing well-defined mental steps required to make an informed conclusion based on the input data.

A hybrid visual-textual language called Thought Process Trees (TPT) was developed to facilitate creating formal models of subject domains. It is named following the terminology introduced by O'Rourke et al. [31]. In edu-

cational applications, Thought Process Trees can describe processes of finding the problem's solution (Solving Trees, which are similar in their properties to rule-based domain models) and verification and grading of a student's solution (Verification Trees, akin to constraint-based models). The reasoning is performed over a formal representation of the learning problem formulation, for example, in the form of a knowledge graph in the Resource Description Framework (RDF) format [39] . A single tree usually represents the reasoning sub-steps necessary to perform (grade) correctly one problem-solving step.

Thought Process Trees consist of several kinds of nodes (see Figure 1:

1. *Question nodes* are inherited from decision trees. They are drawn as ovals as shown in Figure 1 (a). The node contains a question; the outgoing edges are labeled with possible answers to that question. The question can be Boolean (i.e., with answer either "yes" or "no"), numerical (with a limited number of answers), or have enumerated answers.
2. *Action nodes* represent actions which assign a value (typically, an object) to a variable during reasoning. They are drawn as rectangles with rounded corners; the new variable is usually displayed at the last line of the node label. The most common actions are finding an object by a certain set of criteria and calculating a new value from existing variables. Action nodes usually have one outgoing edge that is not labeled; search nodes can have a second edge labeled "Not found" if it is possible that no object satisfying criteria can be found. Action nodes in Hinting Trees can have special outgoing edges shown by dashed lines representing possible errors while performing an action. See the example in Figure 1 (b)
3. *Outcome nodes* contain actions that must be executed in the knowledge graph depending on the results of the reasoning. As they contain actions, they are also drawn as rectangles with rounded corners but have a green colour. In Verification Trees, outcome nodes can be not only green (correct step) or red (incorrect step), as shown in Figure 1 (c). Outcome nodes do not have outgoing edges.
4. *Branching nodes* begin independent branches of thought that should be performed (verified) to find the correct solution. They are displayed as hexagons. The outgoing edges for Branching Nodes are labeled with branch names. There are two kinds of branching nodes.
   (a) *Aggregated branching* is used in Verification Trees; it aggregates conclusions from its branches using a Boolean operator (usually,

`AND`). They do not have separate names or formulas; their function is purely structural. See Figure 1 (d).

   (b) *Independent branching* is used when a set of conditions must be verified independently, and in any possible situation, only one of them can give an outcome. It can have a special outgoing edge labeled "Undetermined," which is taken if no branch gives an outcome. See Figure 1 (e).

5. *Iteration nodes* represent iterating through a set of objects, repeating the same reasoning steps for each of them. They are also displayed as hexagons but contain universal quantifiers as the sign of iteration or the words "while" or "until". Iteration can be done over objects satisfying certain criteria or until some condition holds. The outgoing edge going down from an iteration node leads to the iteration body, while the edge to the left or right represents actions taken after the iteration ends. See Figure 1 (g).
6. *Recursive nodes* represent rare situations when performing a task step requires applying the same lines of reasoning to other objects. They are typically used after branching to avoid eternal recursion. Recursive nodes are drawn as two circles; see Figure 1 (f).

The tree is accompanied by dictionaries that explain the data model over which the tree is executed. A thought process tree requires defining several dictionaries (see Table 1 for example entries for each dictionary):

1. *Classes* of objects, which can be related by inheritance ("is a") relationship. Classes determine the kinds of objects that constitute the problem formulation and solution. Usually, a new class should be added when it is necessary to restrict some property or relationship to the objects of that class.
2. *Class properties* are data properties whose values are the same for all the objects of the given class. They are usually used in parent classes so that child classes could have different values.
3. *Object properties* are data properties that can hold different values for the objects of the same class. Both class and object properties can have several different types:
   - Boolean properties hold values yes or no;
   - numerical properties hold numbers that can be limited by specified minimum and maximum values;

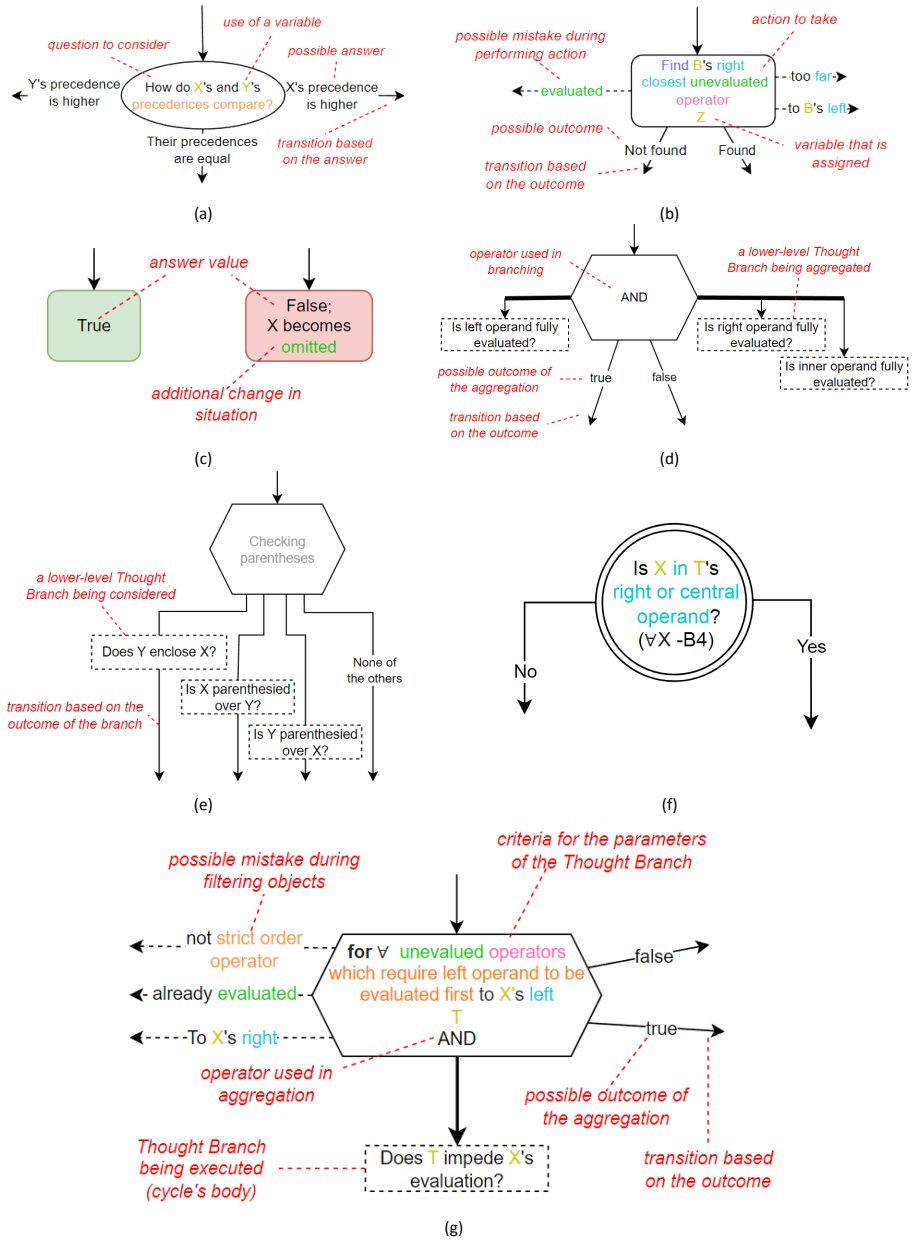


Figure 1: Node types in Thought Process Trees: (a) Question node, (b) Action node, (c) Outcome nodes, (d) Aggregated branching, (e) Independent branching, (f) Recursion node, (g) Iteration node.

- enumerated properties can hold values from the specified lists;
- linear scale properties can be compared as "lower", "equal" or "higher".

4. *Relationships* connect objects. A relationship has a name and at least two objects (restricted by classes) that can participate in it. Common kinds of relations produce several calculable relationships from the basic relationship, for example:
   - Partial order relationship is based on the relationship "is directly less" and produces relationships likes "is less", "is more".
   - Linear order relationship is based on the relationship "is directly less" (which, according to the situation, can be written as "directly to the left", "directly below" and so on) and produces the same kinds of relationships (e.g., "to the left", "to the right"), the relationship "between", and relationship "closer to" with the extreme form "closest".
5. *Questions* dictionary lists the kinds of questions used in the particular tree.
6. *Actions* dictionary lists the kinds of actions used in the particular tree. These two dictionaries are useful for unification to avoid creating several similar questions and actions.

Hinting trees describe the process of solving a learning problem or a step of that process. All their outcome nodes are green; they can have textual explanations attached. They can be used for the following tasks:

1. solving learning problem given the formal problem formulation and finding a correct answer, which can be used both for human-entered problems and is a necessary part of automatic learning-problem generation;
2. showing worked examples with textual explanations;
3. showing hints during problem-solving;
4. maintaining educational dialogue by generating pedagogical questions about the declarative and procedural knowledge required to solve the problem step.

Hinting trees are similar in their functionality to TPL programs [31], but hinting trees offer several advantages in describing the problem-solving

Table 1: Examples of dictionary entries for a Thought Process Tree

Entry kind	**Entry example**
Class	Element
Inherited class	Operator (is an element)
Boolean property defined in the subclasses of the class operator	Needs left operand (operator): bool
List property defined in the objects of the class element	State (element), list: unevaluated, evaluated, used, omitted
Relationship between tokens and elements (with the reverse relationship in parentheses)	<element> has <token> (<token> belongs to <element>)
Linear order relationship between tokens	<token1> is to the left of <token2>
Three-object calculable relationship	<token1> is between <token2> and <token3>
Proximity relationship	<token1> is closer to <token2> than <token3>
Extreme form of proximity relationship	<token1> is closest to <token2>
Question with integer answer	How many <property name> <class name> has?
Question with a list of possible answers	Compare <property> of <object1> and <object2>: less, equal, more
Action	Find <constraint expression> <class>

process. The crucial difference is that TPL requires specifying an algorithm with a strict order of performing its statements, while Thought Process Trees allow avoiding specifying an order of actions when it does not matter. A typical situation in educational modeling is performing several checks when the order of their verification does not matter, either because their results are aggregated or only one of them (at most) is expected to be true in any circumstances. Thought Process Trees — the modeling language presented in this article — can specify that using Branching nodes, while TPL relies on typical programming-language syntax with the order of branch checks strictly specified in the program. The same is true for iteration when the order of iteration is not important (for example, when you need to check a number of objects satisfying certain conditions): the proposed model allows specifying any possible order, while TPL will specify a specific order of iteration. It is not important in some applications (like problem-solving and demonstrating worked examples) when any correct solution is acceptable. However, it is important for the applications where students' solutions are compared to the tutoring system's solution because a TPL-based tutoring system will insist on having only one correct way of doing things and mark correct solution steps as wrong. That makes Thought Process Trees more suitable than TPL for modeling domains with big solution spaces and many possible solution paths.

Verification trees describe a process of verifying the correctness of a student's solution. They support both green (no errors in this branch) and red (an error is identified) outcome nodes. Those trees can be used in intelligent tutoring systems for the following tasks:

1. verifying students' solutions and their steps;
2. generating error messages explaining why the particular step is wrong;
3. maintaining pedagogical dialogue by generating pedagogical questions about the knowledge necessary to solve the learning problem to help the student realize where they are wrong.

Verification trees can also be used to solve the learning problem. However, they are often ineffective; their performance is comparable to the performance of brute-force search and backtracking methods, which perform poorly for many tasks.

One useful feature of Verification trees is that a correctly built verification tree provides the natural classification of errors during the learning-problem solution (step), with each red outcome node corresponding to a different

kind of error. The path to the red outcome node will assign all the variables required to generate the error message. Verification trees can be simulated as Boolean functions in TPL, but I am not aware of any attempt to do it.

The idea of formalizing the reasoning processes in ITS that led to developing the model of Thought Process Trees was first introduced as "lines of thought" by Kamennov et al. [40] in a paper describing generation sets of low-level reasoning errors during problem solving. Its enhanced version was described in [32] along with the method of generating pedagogical questions based on it. Those papers dealt with particular applications of the Thought Process Trees formalism, but they never touched on the problems of developing Thought Process Trees or the dictionaries required to run Thought Process Trees using formal reasoners. However, when developing Thought Process Trees, the scientific team I led encountered many problems. Developing hinting trees was simple and straightforward because describing the process of problem solving is a well-studied field, and for learning problems with many solutions, only one solution must be described. Developing verification trees proved to be much more challenging. Some (very few) kinds of learning problems were modeled very easily, but many subject domains required several iterations of completely redrawing verification trees before the well-working variant was created. There were different methods of organizing the tree and identifying errors. Also, that situation was caused by the fact that when verifying the student's solution and/or providing step-by-step feedback on their thinking, you must model every possible method of solving the learning problem. All-in-all, modeling possible errors while performing the task is a much less developed field than modeling performing the task itself. This article describes two classes of learning problems that influence the difficulty of building verification trees and shows on examples several methodological principles of their building that increase the quality of verification trees and feedback generation using them, which were not published in previous literature and can significantly reduce the time and effort required to model learning-problem verification.

Modeling problem verification is arguably a more important part of designing intelligent tutoring system as the approach to verification determines the kinds of explanatory messages the student gets. However, this area received much less attention than modeling problem-solving. One of the goals of this article is to fill that gap in the literature. In the next section, practical examples of modeling solution verification using Thought Process Trees will be considered.

4. Modeling Solution Verification

Modeling a student's solution's verification is non-trivial because it constraint-based verification is a kind of declarative modeling [41] while the underlying knowledge on solving the task is procedural. The model author needs to create a set of constraints (check) and the tree of their usage so that every wrong solution will break some constraint while every correct solution finds them all. Constraints can easily model some learning problems, but for many other models, it is a complicated process with many possible solutions, some much less effective than others.

Another critically important question is what should be the input for the constraints in the Verification trees. In a recent review [42], the authors note that the input for constraint-based models describing the learning problem is a correct answer (possibly one of many correct answers), which can be entered by a human teacher authoring the exercise or obtained by an automatic solver. While those constraints are relatively easy to develop, they are of limited pedagogical usage because they do not rely on the formal problem formulation and do not refer directly to the objects of the problem formulation in the generated explanatory messages. When using a correct answer as input to constraints, the resulting "explanatory" at least some error messages look like "Your answer is wrong because the correct answer to this learning problem must contain the following..."

The other approach is used in the CompPrehension intelligent tutoring system [43]. It involves using the formal problem formulation as input to constraints; the constraints must reflect the problem-solving rules of the subject domain. It requires more work and, for now, allows for modeling simpler learning problems. However, the resulting explanatory messages directly reference the problem formulation and describe why the student's answer is wrong in these particular circumstances, which is important for contextualizing knowledge structures in the brain and learning better [3].

The visual representation of node text in Verification trees shown in this section will use the colour coding shown in Table 2.

4.1. Simple case: constraint-based learning problem

The easiest class of learning problems to model using Thought Process Trees are constraint-based learning problems. These are the problems which have no better solving strategy than iterating over the objects mentioned in the problem formulation and including in the answer the object satisfying

Table 2: Colour coding of text in the Verification trees

Colour	**Denotes**
Magenta	Class
Orange	Class property
Green	Object property
Cyan	Relationship
Blue	Question and action keywords

certain conditions (constraints). The inclusion of some objects can open the way to the inclusion of previously forbidden objects, so the process must be repeated until no object can be added to the solution. Note that many well-defined problems [44] can be solved by that strategy which is basically brute-force search, but for constraint-based problems, it is the optimal strategy by definition.

Constraint-based problems often require putting objects in a certain order. Despite their restrictions, they cover some important classes of learning problems. Two notable examples of constraint-based problems are the following problems.

1. Order of expression evaluation (in programming and mathematics). The problem formulation is the linear order of tokens, which can be operands and parts of operators. The answer is the order in which the operators can be evaluated. The constraints define which operators in which circumstances must be evaluated first. While any particular answer is a linear order, the set of all the correct solutions can be represented as a partial order relation "must be evaluated before" between operators. An example of constraint: if two binary operators are in linear sequence without parentheses between them, the operator with the highest precedence must be evaluated first.
2. Word order in an English sentence or phrase. The problem formulation can be presented as a dependency tree [45] — a set of head-dependent relationships between words labeled with their syntactic function. The answer is linear word order, determined by the semantic relations in the tree, grammar rules, and, sometimes, word attributes (for example, the order of adjectives is determined by their respective classes). An example of constraint: an adjective belonging to a noun must stand

before the noun but after the determiner belonging to that noun.

The rest of this section will use the order of expression evaluation problem as a simpler and better-defined problem that allows a manageable size of the Verification tree. Tables 3–6 present the dictionaries required to formally define the problem and its formulation. To formally define the problem, two different fundamental classes of objects are necessary: tokens (the smallest meaningful elements of the language) and elements (which can be operands and operators). Tokens are written linearly, forming the visible side of an expression by the relationship "stands directly before" ("stands directly to the left of"). Operator elements are evaluated, but they are not tokens (some operators, like brackets, consist of two tokens) — they are separate objects linked to tokens by the relationship "has". Operators and operands form relationships "is operand of" (which forms a tree — a hierarchical relationship) and "must be evaluated before" (which forms a direct acyclic graph, enhancing the tree because of operators that evaluate their operands in a specific order). Distinctions between similar but formally different objects like tokens and operators are important in formal modeling.

There are a lot more class properties than object properties because class properties store subject-domain facts necessary to solve the learning problem, while object properties store the solution's state during the process of problem solving. Note using linear scale property type for storing operator precedence instead of an integer number. While each operator in the programming language has a certain level of precedence, which can be expressed as a number, the student, typically, is not expected to remember the numerical values of operator precedence — they only need to compare the precedence of any two operators. The data model respects, so the tutoring system will not demand knowledge of exact operator precedence.

The Verification Tree describes verifying a single problem-solving step — determining if the operator, selected by the student, can be evaluated right now. It must aggregate conclusions of four independent branches, answering the following questions.

1. Can the central operand prevent this operator from evaluation?
2. Can the left operand prevent this operator from evaluation?
3. Can the right operand prevent this operator from evaluation?
4. Are there operators with a strict order of operands evaluation preventing this operator from evaluation?

Table 3: Classes for the order of evaluation thought tree

Class	**Base Class**	**Comment**
Token		Tokens form the expression's notations
Element		Elements of expression that can have a value
Operand	Element	Passive element of expression. Starts in the evaluated state.
Operator	Element	Active element of expression. Starts in the unevaluated state. Each particular operator (e.g., assignment, substraction) is a class inherited from the Operator class.

The conclusions of these four branches are aggregated using the logical AND operator.

The central operand branch is only actual for the two-token operators and is the shortest: it verifies that every operator inside a two-token operator is evaluated before the enclosing operator. It can be seen in Figure 2 The branches for the left and right operands are similar except for the effects of associativity, which are reversed. They require looking for the competing operator (the closest unevaluated operator) to the operator selected by the student and then checking three possible conditions that can determine the outcome (in the following order):

1. parentheses (or enclosing operator) that separate the two operators in question;
2. comparing precedence of the two operators;
3. using associativity to determine the order of evaluation.

There are three cases of parentheses or similar operator being involved, and only one of them can be true, so the order of their verification does not matter; they are verified using an independent branching node. Figure 3 shows the branch for checking the left operand.

The fourth branch concerns operators, which create sequence points in expression evaluation and may omit evaluation of some of their operands depending on the value of their left operand (for example, the operators &&, ||, comma, and ? : in the C++ programming language). That branch — see Figure 4 — is more complex, requiring two nested iterations (iterating all the

Table 4: Class properties for the order of evaluation thought tree

Property	**Base Class**	**Type**	**Limitations**
Number of tokens	Element	int	1 or 2
Arity	Operator	Enumerated	unary, binary or ternary
Placement	Operator	Enumerated	prefix, postfix, infix or enclosing
Needs left operand	Operator	bool	
Needs right operand	Operator	bool	
Precedence	Operator	Linear scale	Linear scale properties can be compared as "more", "less" or "equal", but the actual value is not significant.
Associativity	Operator	Enumerated	left, right or non-associative
Requires the left operand to be evaluated first	Operator	bool	
Required value of the left operand to evaluate the right operand	Operator	Enumerated	true, false or any
Required value of the left operand to evaluate the central operand	Operator	Enumerated	true, false or any

Table 5: Object properties for the order of evaluation thought tree

Property	**Base Class**	**Type**	**Limitations**
State	Element	Enumerated	unevaluated, evaluated, used, or omitted
Evaluates to	Element	Enumerated	true, false or not known

Table 6: Relationships for the order of evaluation thought tree

Relationship	**Base relationship**	**Comment**
<element> has <token>		A basic relationship connecting elements and tokens
<token1> is directly to the left of <token2>		The basic relationship between tokens, a linear order
<token1> is to the left of <token2>	directly to the left	Transitive version
<token1> is between <token2> and <token3>	directly to the left	Has a variant: <token1> is in parentheses <element>
<token1> is closer to <token2> than <token3>	directly to the left	Extreme form: <token1> is closest to <token2> to the left/right
<element> is <operator>'s operand		The basic relationship between elements
<element> is in <operator>'s operand	element is operand	Transitive version

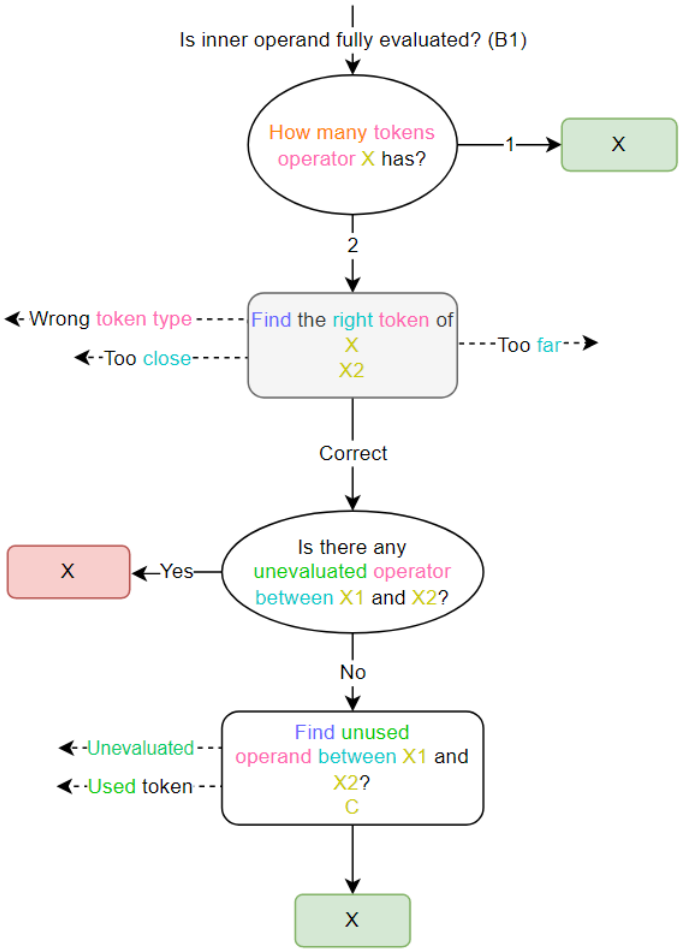


Figure 2: Branch for verifying if the central operand prevents the operator X from evaluation.

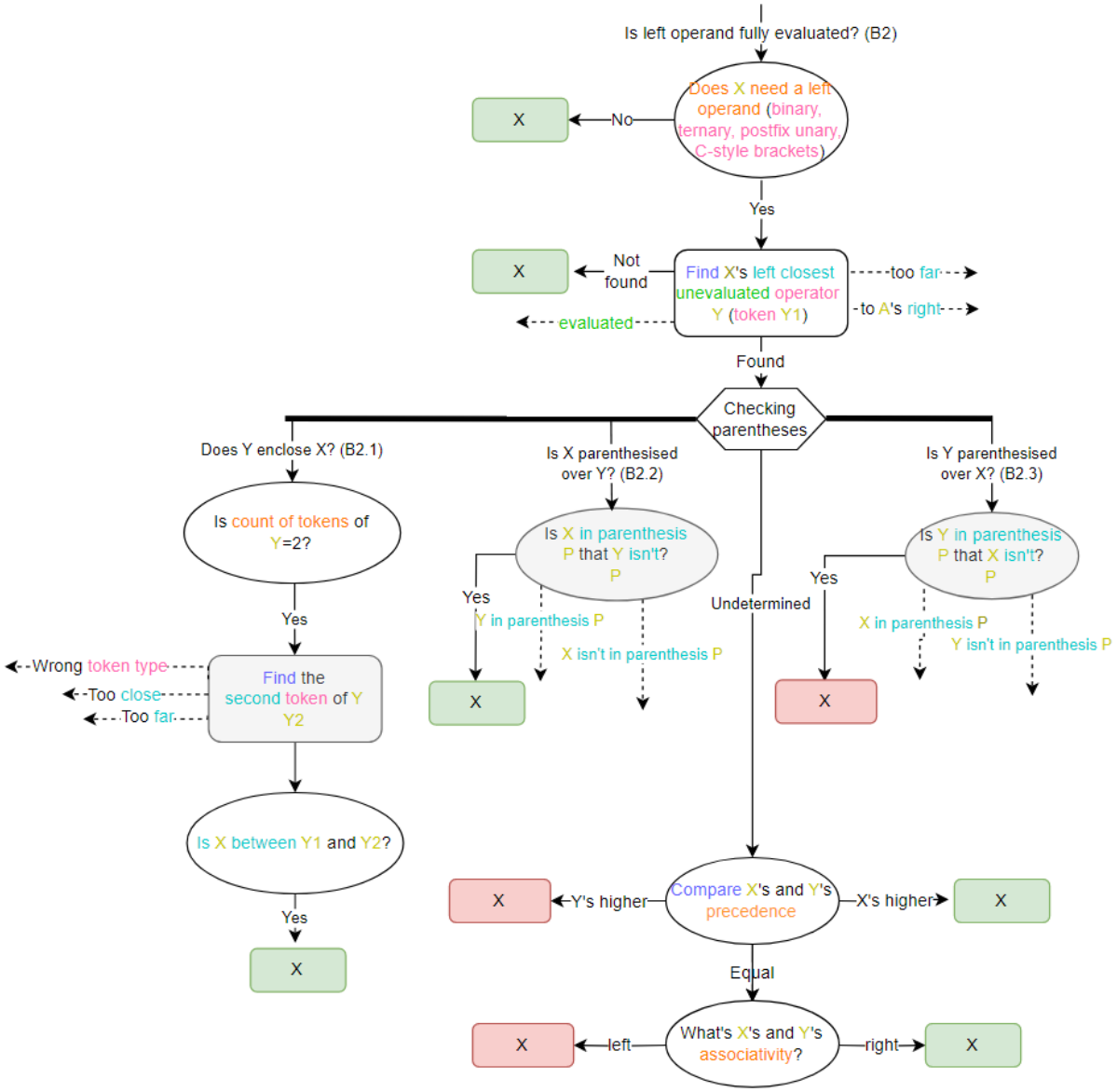


Figure 3: Branch for verifying if the left operand prevents the operator X from evaluation.

operators with the strict order of operand evaluation and the operators in their left operands) and recursive nodes, requiring students to solve the same learning problem for a subexpression to determine which operator belongs to which operand. This recursion is formally possible because, unlike the three other branches, the fourth branch does not affect the "element is operand" relationship used in the recursive nodes. It also shows post-iteration processing for the inner loop.

The tree was used to create an intelligent tutor with explanatory feedback concerning students' error and hints of correct actions [46].

4.2. Lessons learned when modeling thought processes for education

The whole tree for the order of expression evaluation was not developed at once: it was refined several times during the development of the relevant intelligent tutoring system [43] to take into account the inefficiencies of the first versions of the tree. In this section, the methodology lessons learned in this process are discussed to provide guidelines on avoiding common modeling mistakes.

4.2.1. Storing arrays versus iteration.

Thought Process Trees, like the other methods of modeling human reasoning (e.g., Thought Process Language, first-order logic calculus, Apache Jena rules, OWL Web Ontology Language and so on), allow storing found or constructed objects in variables. However, the new variables assigned during tree execution model human short-term memory during problem solving. Short-term memory is viewed as distinct from long-term memory [47] and very limited capacity [48]. Holding an array of objects in short-term memory and then performing some actions for each of them is untypical for human reasoning. Instead, the reasoning requiring verification of a set of objects can be performed one by one — that is, processing each object as soon as it is found and then forgetting it when processing the next object. So, when modeling a thought process using Thought Process Trees, generating an array of objects and storing them in variables is forbidden; otherwise, the tutoring system will require the student to remember the entire array while dealing with its elements. Instead, iteration nodes that iterate over a set of objects satisfying certain conditions are provided (see Figure 4). Sets of objects are only permissible as a part of the problem formulation and answer because they are directly visible to the student during problem-solving in a tutoring system and should not be kept in short-term memory.

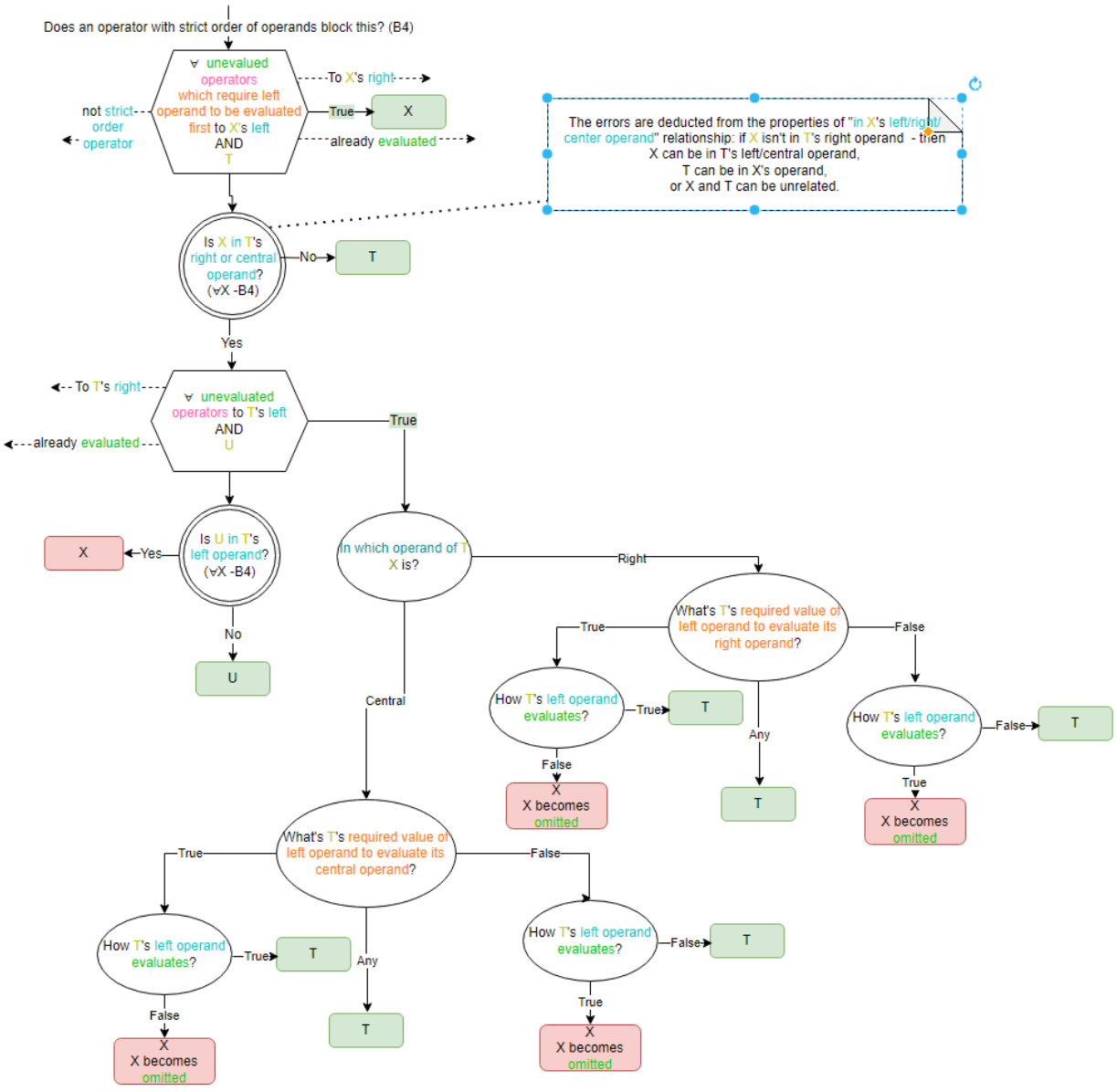


Figure 4: Branch for verifying if any operator with a strict order of operands evaluation prevents the operator X from evaluation.

4.2.2. Sequence of questions versus independent branches.

A typical situation during verification of the student's solution is verifying several criteria in a row. In a typical programming language, the branching operator specifies the order of branches and their conditions are verified in the order they are written; the first true condition wins. TPL developed by O'Rourke et al. [31] is based on the object-oriented programming paradigm and uses the same approach. While that approach is good when it is enough to specify just one way of solving the problem correctly, it does not work in advanced tutoring applications when the software verifies the student's line of thought (for example, by using sequences of pedagogical questions [32]) because it will make the software insist that there is only one way to reason correctly.

Consider the two sets of checks performed during determining if the left operand is fully evaluated. After finding the competing operator to the left, to determine which operator must be evaluated, the first three conditions are checked:

1. if parentheses or enclosing operator affect the order of evaluation;
2. precedence of the two operators;
3. associativity of the two operators (which is, by definition, the same if they have the same precedence).

In this case, the order of verification matters: parentheses override everything else when they are involved, and associativity is only taken into account if the operators have the same precedence. So, using subsequent questions as shown in Figure 3 makes sense.

To verify if parentheses play any role in the respective order of evaluation of two found operators, it is also necessary to assess three conditions:

- if the left operator is in the parentheses, while the current operator is outside of them;
- if the current operator is in the parentheses, while the left operator is outside of them;
- if the left operator is an enclosing operator, and the current operator is inside it.

However, for these conditions, the order of checking is not important because only one of them can yield the result that gives a certain outcome. While

two operators can be in different sets of parentheses, there must be a third operator between them that should be evaluated after them so they will not be competing operators under any circumstances. During naive modeling, it is common to have these conditions chained, as shown in Figure 5 (a). To avoid specifying the order, which is not necessary, and so avoid failing correct reasoning, the independent branching node should be used instead, as shown in Figure 5 (b).

4.2.3. Questions about specific classes versus questions about class properties.

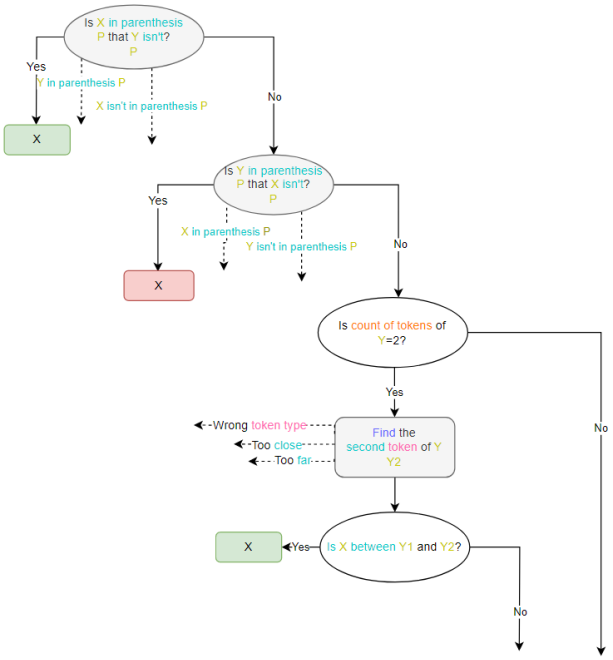
Many subject domains have classes with unique properties. In that case, it is tempting for the tree's author to formulate questions using class names. I.e., "is the animal a cow?" or "Is the statement a FOR loop?". However, questions like that are not very useful for learning (e.g., when generating error explanations and pedagogical questions) because they link knowledge to the class name instead of introducing the idea that there are properties shared by objects of the same class.

For example, one of the programming languages with the most developed and complex expression system is the C++ programming language. It served as the basis for many modern programming languages. Most of the operators in expressions do not specify the order of evaluation of their operands, but there are exceptions. The C++ language has four operators with a strict order of operand evaluation, which perform differently:

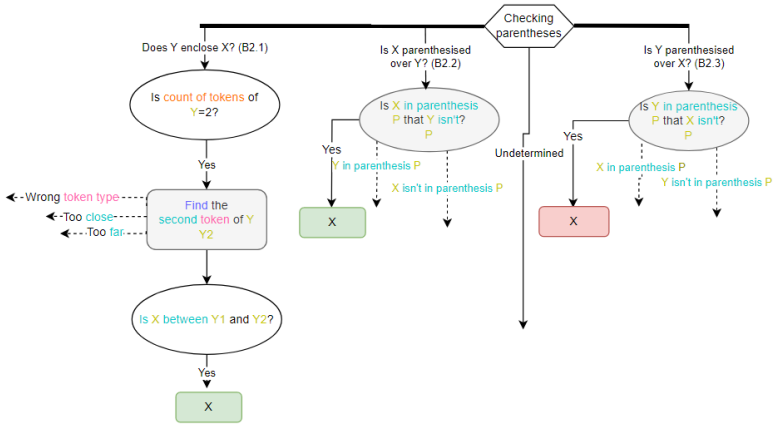
- the comma operator simply evaluates two operands in the order they are listed and returns the value of the last operand;
- the logical AND operator evaluates its right operand only if its left operand evaluates to true;
- the logical OR operator evaluates its right operand only if its left operand evaluates to false;
- the ternary operator evaluates its central operand if its left operand evaluates to true and evaluates its right operand if its left operand evaluates to false.

Some of the other programming languages have different behaviour of these operators or lack some of them altogether.

The naive approach to modeling this situation is asking about the type of operator in question nodes, as shown in Figure 6 (a). While that tree will



(a)



(b)

Figure 5: A set of sequentially added questions (a) versus independent branching (b).

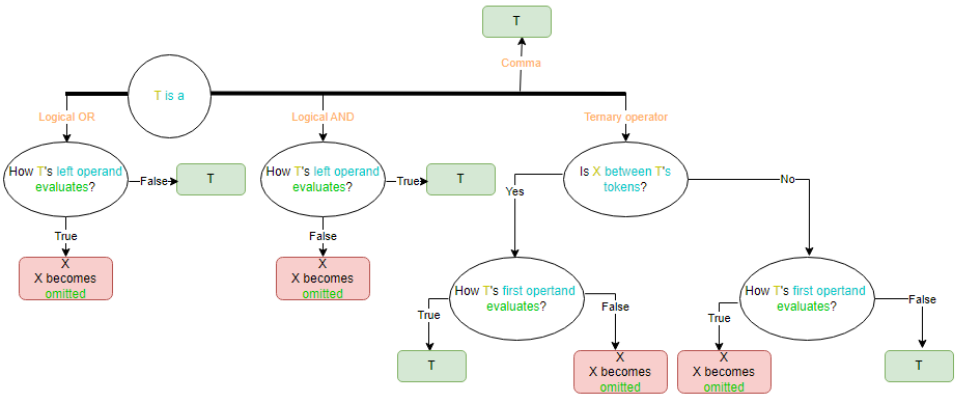
work, it limits the number of programming languages it describes; supporting some of the languages will require modifying the tree. What is worse, it makes the tutoring system teach the student that "that operator always works like that" as if it was dependent only on the kind operators. In practice, in different programming languages, the same operators can have different properties, so it is better to teach students about that something that can be changed.

The alternative approach that I find more efficient is introducing class properties like "Required value of the left operand to evaluate the right operand." Instead of simply concentrating the student's attention on the behaviour of a particular operator in a particular language, it brings to the student the general idea that "there are operators which behave like that," considering it an operator's property. Then, the automatic tutor can concentrate on the question "which operators in that programming language have this feature." That helps students develop a more flexible knowledge framework than linking objects' behaviour directly to their classes. Figure 6 (b) shows the fragment of the Verification tree for this approach.

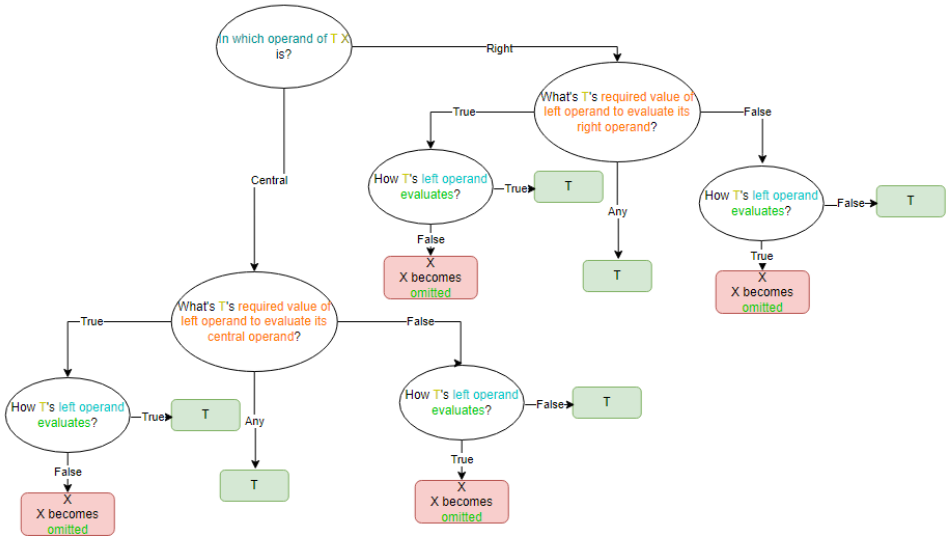
4.3. Tricks of explaining errors: tasks with one correct answer

The task of modeling possible errors and generating explanations of why they are wrong becomes more complex when modeling learning problems with only one correct answer each. There are many learning problems like that in well-defined domains like mathematics, programming and sciences. In that case, the initial attempts of new members of the research team developing and using Thought Process Trees always led to developing Hinting trees — that is, describing the process of finding the correct answer.

While learning correct answers from hints is enough for some students to master the topic, the lack of explanation of why their answers were wrong can interfere with learning because it leaves generalization and contextualization of knowledge to students' imagination, which often ends up in learning by trial-and-error [7]. The "explanatory" error messages generated by modeling reasoning like "your answer is wrong because the correct answer in these circumstances is..." While it gives some information, it does not explain what was wrong with the student's answer. So, after experimenting with different kinds of learning problems [43], I came to the conclusion that Verification Trees and constraint-based approach must be extended to work on tasks with a single correct answer.



(a)



(b)

Figure 6: Question directly mentioning classes (a) versus questions regarding class properties (b).

4.3.1. Task solving versus constraints.

Consider examples of sub-trees, shown in Figure 7. They model the task of determining the next action to execute for the simplest algorithmic structure: sequence. The left part shows the result of the hinting approach: it finds the last executed statement and calculates which one will be executed next; the explanatory message for errors, in that case, will look like the general example in the previous paragraph. The right part shows the constraint-based approach, representing the same problem as two independent branches verifying different constraints:

- the statement must not be already executed (during this execution of the sequence) — the error message for violation is "your answer is wrong because the statement you selected was already executed, and the sequence structure executes each statement only once";
- the executed statement should not have other non-executed (during this execution of the sequence) statements before it — the error message is "your answer is wrong because the statement X was not executed yet, and it stands before the statement you selected; the statement X must be executed first".

The explanatory messages for the constraint-based model contain more contextualizing information than hints explaining the correct answer, which can help students learn [3]. Instructions for achieving the correct answer are usually contained in textbooks and lectures, so the students who learn well because of them will make fewer errors in tasks like that.

4.3.2. The order of verifying constraints.

In the practice of developing and evaluating Thought Process Trees, I found that there are two different ways to verify students' answers by using constraints with different types of resulting error messages.

1. Starting from analyzing the current problem-solving situation and finding the context determining the correct answer. That is an intuitive approach because it is close to human problem-solving skills. However, it still relies on the correct answer in the explanatory messages because the logic of identifying the error types in that approach comes from the logic of seeking the correct answer that is compared to the student's answer to the last question. It again leads the tutoring system

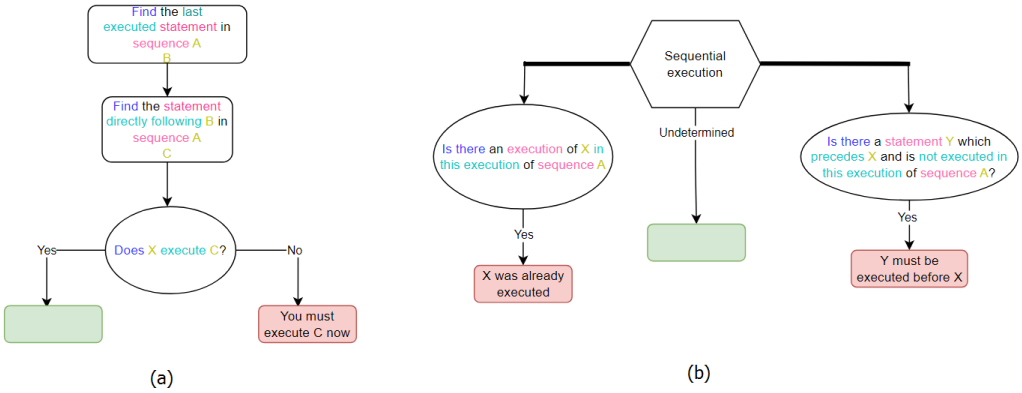


Figure 7: Comparing to the correct answer (a) versus verifying constraints (b). X is the statement, selected by the student. Note the mode detailed error reporting in the right tree.

to generate error messages like "your answer is wrong because in these circumstance the correct answer is different.

2. Beginning from analyzing the student's answer and verifying conditions (from the broadest to narrowest conditions), which can allow the student's answer to be correct. That way, the reasoning stops at the broadest constraint, indicating that something is wrong with the student's answer. It gives more natural error messages, but this approach is often counter-intuitive for the people modeling the subject domain.

Figure 8 demonstrates two ways to verify the selection algorithmic structure using Thought Process Trees. The left part shows a part of the Verification tree, the intuitive approach where the primary variable is the previous correctly executed statement. The automatic tutor first identifies that the student's trace line is inside the selection statement, then it analyzes the previous step — is it the selection beginning, verification of some condition or a branch end (the branch itself is the nested sequence structure and is not considered a part of the selection execution) — and then, if necessary, analyses the result of the verifying the condition of that branch. Only the last question nodes in each branch ask for the student's answer and give a pass to the correct one. That is a direct mapping of the "find correct answer first" approach, which gives less informative error messages.

The right part of Figure 8 shows a part of the Verification tree built using the student-answer-based approach. It starts by verifying that the trace line

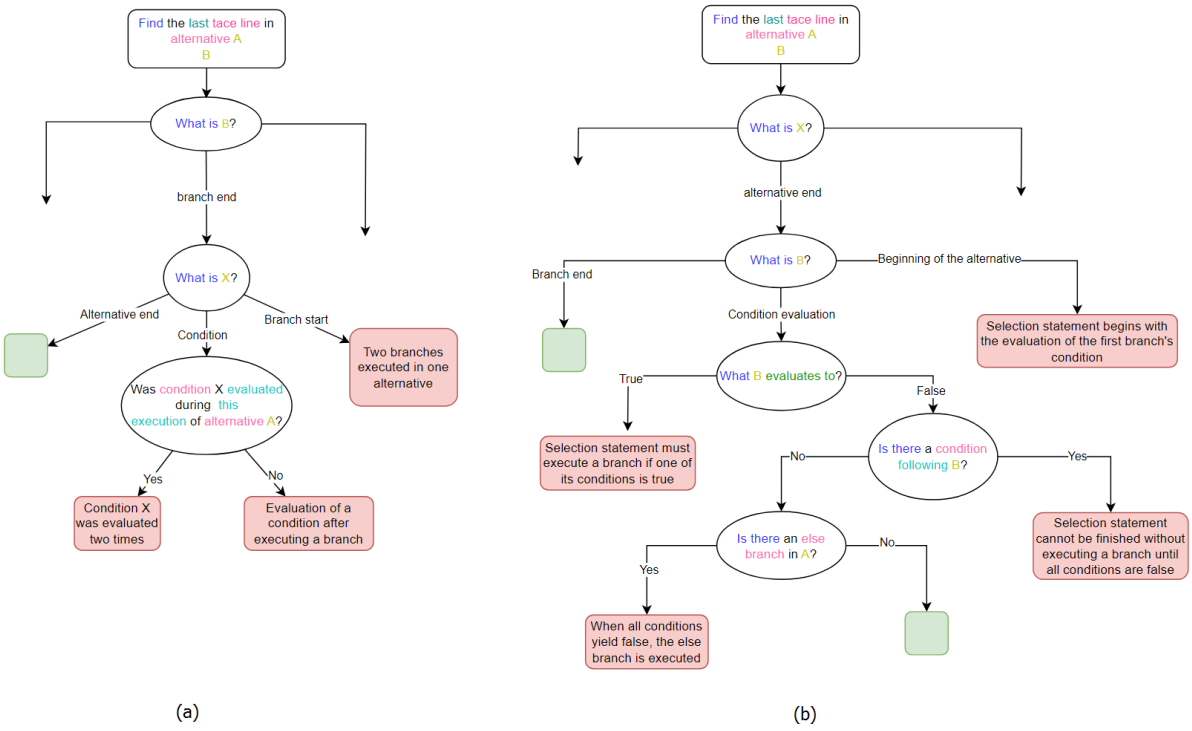


Figure 8: Fragments of the tree for analyzing alternative algorithmic structure (selection statement) using two methods: the intuitive approach beginning with analyzing the current situation (a) and analyzing the student's answer first (b). X is the statement, selected by the student. Note the different kinds of errors detected.

is inside the same algorithmic structure that the student's statement belongs to, and then the checks depend on the kind of trace line the student selected. If it is a branch condition, it verifies the same constraints as for the sequence structure (because the branch conditions are evaluated sequentially until the first condition, which evaluates to true) plus the value of the previous condition. If the student selected the beginning of the branch, it verifies if the branch condition was evaluated and is true. If the student selected the end of the selection statement, it verifies if a branch ended or all conditions were evaluated to false and there is no "else" branch. That gives a more balanced tree structure with conclusion nodes not grouped at the end of the branch and better explanatory error messages like "finishing the selection right now is wrong because, while all the branch conditions were false, it has an else branch which must be executed."

When developing trees for the program tracing subject domain (which includes control flow statements and function calls), an experiment was conducted by creating three different trees:

1. Hinting Tree let find errors of 23 different types;
2. Verification Tree based on the naive approach let find errors of 54 distinct types;
3. Verification Tree based on the analyzing student's answer first approach allowed to find errors of 66 different types, giving the most detailed view of the possible reasoning errors and so more informative error explanation messages.

So, I recommend the second approach to modeling learning problem verification: starting from hypothesizing that the student's answer can be correct and seeking rules that prove otherwise. It works better with the constraint-based approach and gives more detailed explanatory messages which are easier to formulate for the model developer. The alternative approach is avoiding diving two questions and merging them into one node with two comparisons. Generally, for correct error identification and generation of pedagogical questions, two questions must be asked sequentially in the model only if the necessity to ask the second question depends on the answer to the first question. However, that is impossible with Question Nodes supporting only one question per node.

5. Formalizing Thought Process Trees

Thought Process Trees with natural-language text in nodes help systematically model subject domains, verify constraint-based models and can be used for instructional purposes. While supporting the development of a systematic set of constraints is very important because constraints are harder to develop and test than regular imperative programs, the true power of Thought Process Trees can only be achieved by formalizing them so that they can be analyzed and executed automatically.

Fortunately, Thought Process Trees have a strict syntax that can be easily formalized. Basically, that requires selecting a formal model for the data on which the tree will operate and formalizing dictionaries, node and edge labels. As the information about the learning problems and their solutions in intelligent tutoring systems often consists of objects, their properties and relationships, the natural choice for storing this information is Knowledge

Graphs. The widely supported format for storing Knowledge Graphs is RDF: there are both storage and access systems and reasoners developed for it. That serves as a basis for formalizing the information in trees.

Executing formalized forms of Thought Process Trees using reasoning engines (compilers or interpreters) based on the data represented as Knowledge Graphs makes modeling using Thought Process Trees a task belonging to symbolic artificial intelligence field [49].

5.1. Dictionaries

Dictionaries show the data structure of the subject domain. In a formalized form, they are converted to domain ontologies, containing the concept hierarchy and other declarative knowledge of the subject domain as it is done during the development of intelligent tutoring systems [50]. Ontologies allow validation of both Thought Process Trees and formal learning-problem formulations using static typing mechanisms. For the problem formulations in the RDF format, the system can control the presence of necessary and optional properties in classes and their objects and perform domain validation for each relationship. The static type control for nodes can be performed for expressions, described in the following subsection.

5.2. Nodes

When comparing Thought Process Trees to a typical programming language, nodes are equivalent to statements, while node labels are equivalent to expressions. Expressions can set criteria for searching and iterating objects (that is, only objects for which the expression evaluates to true are iterated) and calculate answers to questions in Boolean, numeric and enumerated form.

Existing reasoning systems for knowledge in the RDF format have developed formalisms for writing reasoning rules and relevant formulas; for example, Apache Jena General Reasoner [51] uses a custom production rule format. However, their expressive capabilities are rather small; writing even relatively simple conditions in Jena Rules often requires a lot of text, which significantly increases the development and debugging time. For example, Jena rules do not directly support negation, logical OR operators, and quantifiers.

To address this problem, an expression language was developed to process knowledge graph data based on the first-order logic which allows using arithmetic and logical operators, universal and existential quantifiers, manipulating class and object properties, manipulating relationships between

objects, retrieving objects linked to known object by specified relationships, and so on. For example, the node "find an unused or unevaluated operator that is inside of operator Y and assign it to the variable X" (where "operator" is a class, "unused" and "unevaluated" are values of the property "state", and "is inside of" is a relationship) can be written as the expression (1).

$$
\begin{array}{l}
find\ X\ \{ \\
\quad \$X\ is\ operator\ and \\
\quad \$X -> isInside(\$Y)\ and \\
\quad (\ \$X.state\ =\ element :: unused\ or\ \$X.state\ =\ element :: unevaluated\ ) \\
\}.
\end{array}
\tag{1}
$$

The expression language supports operators for universal and existential quantifiers and extreme searching (for example, searching the closest object with the given relationship).

In practice, learning a new language for developing domain models can be an additional workload. The editor for Thought Process Trees includes a visual expression editor using the popular Blockly library [52], which allows for avoiding any syntax error in the new language.

5.3. Executing Thought Process Trees

The trees can be executed over a given RDF file containing the problem formulation, partial student's solution and the last step. The result is written in the RDF file; for the Verification tree, the presence of errors and exact error outcome nodes are also reported.

Currently, the research team on intelligent technologies in education of Volgograd State Technical University developed under my supervision program code for interpreting Thought Process Trees that was verified while developing and running several intelligent tutoring systems for teaching programming basics and English natural language (see the next section for the list). It performs smoothly but is significantly slower than the reasoners using simpler rule models like Apache Jena reasoner. The interpreter has demonstrated the viability of the Thought Process Trees approach to developing intelligent tutoring systems. Compiling Thought Process Trees into rule languages of modern software reasoners is also possible.

6. Usage of Thought Process Trees

To verify and rectify the model of Thought Process Trees, the research team at Volgograd State Technical University developed several subject domain models, which were used to build intelligent tutoring systems CompPrehension and HowItWorks and a number of prototypes of intelligent tutoring systems. The following subject domains were modeled and used:

1. *order of expression evaluation* — combined Verification and Hinting tree, integrated into intelligent tutoring systems;
2. *program tracing* — Hinting and Verification trees, integrated into intelligent tutoring systems;
3. *variable scope and lifetime* — Hinting and Verification trees, a prototype of tutoring system developed;
4. *access expressions* — Hinting and Verification trees, a prototype of tutoring system developed;
5. *word order in noun phrases of English language* — combined Verification and Hinting tree, a prototype of tutoring system developed and evaluated with students, in the process of integration into the HowItWorks system.

The approximate development time for these trees was about 10–50 person-hours per tree, which is several times lower than manual development and debugging of sets of Jena rules for the same subject domains: for example, the development and debugging Jena rules for the order of evaluation learning problem took 150 person-hours, while the equivalent verification tree took only 20 person-hours.

The developed trees were used for the following tasks:

1. defining sets of meaningful errors for the given kinds of learning problems [40];
2. per-step finding errors in the students' answers and generating explanatory error messages [46];
3. generating hints to the next correct steps with explanatory messages when the student is stuck [46];
4. maintaining dialogue by asking pedagogical questions and commenting on answers to them when the explanatory messages were too difficult to understand [32];
5. solving automatically generated learning problems [53];

6. classifying automatically generated learning problems according to the knowledge necessary to solve them [53].

This experience proves that Thought Process Trees are a viable way for subject-domain modeling; they can significantly speed up the development of constraint-based intelligent tutoring systems.

7. Conclusion and future work

This article described a new way of modeling thought processes for developing intelligent tutoring systems — Thought Process Trees. It is a visual language which can be converted to text form if necessary and supports both natural-language (for human readability) and formalized (for automatic execution) labels. Thought Process Trees have advantages over similar formalisms like Thought Process Language [31] because they support verifying independent sets of conditions and iterating objects satisfying certain criteria without specifying the order when it is unimportant, giving the student more flexibility in learning.

Several recommendations for modeling reasoning for education were developed, in particular:

- do not store arrays in variables; use iteration instead to avoid overloading students' short-term memory;
- use independent branching instead of chaining questions when the order of the questions is not important;
- introduce class properties defining the behavior of particular classes of objects instead of verifying for these classes directly in the tree;
- use constraints over the student-visible problem formulation instead of comparing to the correct answers even if there is only one correct answer to each learning problem to make error explanatory messages deeper;
- when building constraint sets, begin them by verifying the student's answer instead of identifying the current situation before the answer.

To prove that Thought Process Trees are a viable modeling language for developing subject-domain models for intelligent tutoring systems, they were

formalized using ontologies and special expression language based on first-order logic; a framework for developing constraint-based intelligent tutoring systems was developed, which was tested and refined during the development of five subject-domain models. The framework significantly decreased the time and effort spent on developing subject-domain models and eased their verification regarding following the described methodology, allowing the team to catch many methodological errors. The framework contains an interpreter of Thought Process Trees; the further ongoing work includes developing a compiler of Thought Process Trees into rules of Apache Jena Reasoner, which will significantly speed up the trees execution.

Ultimately, modeling reasoning during problem-solving for learning purposes leads to modeling human thinking because a sound intelligent tutoring system must perform the reasoning that is expected of the student, find the places where the student deviated from the expected reasoning (including in a dialogue with the student), and offer corrections. So, the models developed initially for educational purposes require teaching computers to think like humans and improve understanding of human cognition and reasoning.

There are several ways of further research in that direction. The first direction is widening the class of learning problems supported by Thought Process Trees. The initial experiments in modeling topics of the English natural language show that this method can be used outside of STEM disciplines. However, more research is necessary to determine the limitations of the developed method, both in subject domains and cognitive classes of learning problems. Currently, Thought Process Trees allows modeling relatively simple intellectual skills requiring reasoning using well-defined rules to the level when a computer can both perform the task and explain to humans when and why they are wrong in performing the task. Can this model be extended to cover emerging features of intelligence like invention and creativity? This is one of the important questions of further research. It is also needed to validate if the tree structure is enough to model all the necessary processes or if the trees should be upgraded to Thought Process Graphs.

An important feature is upgrading Thought Process Trees for dealing with learning problems where students' answers do not contain enough information to make a decision about which error the student made (or if it was an error in their thinking). The ITS needs to learn to form hypotheses about the students answers and validate them in a dialog with the student.

Another significant avenue for further research is creating blended models using machine-learning techniques. They can be useful in enhancing machine

replies in the dialog with the student (that is, the symbolic AI part generates the content of the message, and an LLM is used to generate a better natural-language message with that content). They can also help with the adaptive selection of the next learning problems to solve based on the student's performance and the semantic errors they make (and does not make). Those tasks do not require precision reasoning and have loosely defined rules, which makes them ideal for applying machine-learning techniques.

More general goals for future research include studying the possibility of automatically building efficient Verification trees based on Hinting trees (the intuitive way of converting trees gives poor explanatory messages), determining the classes of learning problems that must be solved to verify that the student mastered the subject domain by analyzing its Thought Process Tree. It will also be used to generate not only personalized feedback but also the learning problems used in intelligent tutoring systems to save the effort of teaching staff on creating big banks of learning problems.

References

[1] D. J. Deming, K. Noray, Earnings Dynamics, Changing Job Skills, and STEM Careers*, The Quarterly Journal of Economics 135 (4) (2020) 1965–2005. doi:10.1093/qje/qjaa021.

[2] B. S. Bloom, M. B. Engelhart, E. J. Furst, W. H. Hill, D. R. Krathwohl, Taxonomy of educational objectives. The classification of educational goals. Handbook 1: Cognitive domain, Longmans Green, New York, 1956.

[3] S. Ohlsson, Learning from performance errors., Psychological Review 103 (1996) 241–262. doi:10.1037/0033-295X.103.2.241.

[4] W. Dejene, The practice of modularized curriculum in higher education institution: Active learning and continuous assessment in focus, Cogent Education 6 (1) (2019) Research Article. doi:10.1080/2331186X.2019.1611052.

[5] M. Amelung, K. Krieger, D. Rösner, E-assessment as a service, IEEE Transactions on Learning Technologies 4 (2) (2011) 162–174. doi:10.1109/TLT.2010.24.

[6] C. Glover, E. Brown, Written feedback for students: too much, too detailed or too incomprehensible to be effective?, Bioscience Education 7 (1) (2006) 1–16. doi:10.3108/beej.2006.07000004.

[7] O. Sychev, Write a line: Tests with answer templates and string completion hints for self-learning in a CS1 course, in: 44th IEEE/ACM International Conference on Software Engineering: Software Engineering Education and Training, ICSE (SEET) 2022, Pittsburgh, PA, USA, May 22-24, 2022, 2022, pp. 265–276. doi:10.1109/ICSE-SEET55299.2022.9794157.

[8] A. Gaeta, F. Orciuoli, A. Pascuzzo, A. Peduto, Evaluating the ability of large language models to generate motivational feedback, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 188–201. doi:10.1007/978-3-031-63028-6_15.

[9] A. Garg, R. Rajendran, Analyzing the role of generative ai in fostering self-directed learning through structured prompt engineering, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 232–243. doi:10.1007/978-3-031-63028-6_18.

[10] A. Fehnker, L. Glover, Keeping humans in the loop: Llm supported oral examinations, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 312–319. doi:10.1007/978-3-031-63028-6_25.

[11] L. M. D. Nguyen, F. Lin, M. Chang, Generating learning sequences using contextual bandit algorithms, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 320–329. doi:10.1007/978-3-031-63028-6_26.

[12] Y. Lee, S. Jeong, J. Kim, Improving llm classification of logical errors by integrating error relationship into prompts, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 91–103. doi:10.1007/978-3-031-63028-6_8.

[13] D. Y. Kim, W. J. Lee, Predicting rough error causes in novice programmers using cognitive level, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 341–350. doi:10.1007/978-3-031-63028-6_28.

[14] C. Papakostas, C. Troussas, A. Krouska, C. Sgouropoulou, A rule-based chatbot offering personalized guidance in computer programming education, in: Lecture Notes in Computer Science, Springer Nature Switzerland, 2024, p. 253–264. doi:10.1007/978-3-031-63031-6_22.

[15] O. Sychev, Combining neural networks and symbolic inference in a hybrid cognitive architecture, in: Proceedings of the 2020 Annual International Conference on Brain-Inspired Cognitive Architectures for Artificial Intelligence, BICA 2020, Eleventh Annual Meeting of the BICA Society, November 10-15, 2020, Virtual Event / Natal, Rio Grande do Norte, Brazil, 2020, pp. 728–734. doi:10.1016/J.PROCS.2021.06.085.

[16] H. L. Burns, C. G. Capps, Foundations of intelligent tutoring systems, Routledge, 1988, Ch. Foundations of intelligent tutoring systems : an introduction, pp. 1–19.

[17] C. Conati, A. Gertner, K. Vanlehn, Using bayesian networks to manage uncertainty in student modeling, User Model. User-Adapt. Interact. 12 (2002) 371–417. doi:10.1023/A:1021258506583.

[18] K. Stacey, E. Sonenberg, A. Nicholson, T. Boneh, V. Steinle, A teaching model exploiting cognitive conflict driven by a bayesian network, in: P. Brusilovsky, A. Corbett, F. de Rosis (Eds.), User Modeling 2003, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 352–362. doi:10.1007/3-540-44963-9_48.

[19] E. Millán, T. Loboda, J. L. P. de-la Cruz, Bayesian networks for student model engineering, Computers & Education 55 (4) (2010) 1663–1683. doi:10.1016/j.compedu.2010.07.010.

[20] T. Käser, S. Klingler, A. G. Schwing, M. Gross, Beyond knowledge tracing: Modeling skill topologies with bayesian networks, in: S. Trausan-Matu, K. E. Boyer, M. Crosby, K. Panourgia (Eds.), Intelligent Tutoring Systems, Springer International Publishing, Cham, 2014, pp. 188–198. doi:10.1007/978-3-319-07221-0_23.

[21] T. Barnes, J. Stamper, Toward automatic hint generation for logic proof tutoring using historical student data, in: B. P. Woolf, E. Aïmeur, R. Nkambou, S. Lajoie (Eds.), Intelligent Tutoring Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 373–382. doi:10.1007/978-3-540-69132-7_41.

[22] Y. Shi, K. Shah, W. Wang, S. Marwan, P. Penmetsa, T. Price, Toward semi-automatic misconception discovery using code embeddings, in: LAK21: 11th International Learning Analytics and Knowledge Conference, LAK21, Association for Computing Machinery, New York, NY, USA, 2021, p. 606–612. doi:10.1145/3448139.3448205.

[23] Y. Huang, N. G. Lobczowski, J. E. Richey, E. A. McLaughlin, M. W. Asher, J. M. Harackiewicz, V. Aleven, K. R. Koedinger, A general multi-method approach to data-driven redesign of tutoring systems, in: LAK21: 11th International Learning Analytics and Knowledge Conference, LAK21, Association for Computing Machinery, New York, NY, USA, 2021, p. 161–172. doi:10.1145/3448139.3448155.

[24] R. S. Crowley, O. Medvedeva, An intelligent tutoring system for visual classification problem solving, Artificial Intelligence in Medicine 36 (1) (2006) 85–117. doi:10.1016/j.artmed.2005.01.005.

[25] V. Aleven, B. M. McLaren, J. Sewall, M. van Velsen, O. Popescu, S. Demi, M. Ringenberg, K. R. Koedinger, Example-tracing tutors: Intelligent tutor development for non-programmers, International Journal of Artificial Intelligence in Education 26 (1) (2016) 224–269. doi:10.1007/s40593-015-0088-2.
URL https://doi.org/10.1007/s40593-015-0088-2

[26] S. Ritter, J. R. Anderson, K. R. Koedinger, A. Corbett, Cognitive tutor: Applied research in mathematics education, Psychonomic Bulletin & Review 14 (2) (2007) 249–255. doi:10.3758/bf03194060.

[27] Y. Long, V. Aleven, Gamification of joint student/system control over problem selection in a linear equation tutor, in: S. Trausan-Matu, K. E. Boyer, M. Crosby, K. Panourgia (Eds.), Intelligent Tutoring Systems, Springer International Publishing, Cham, 2014, pp. 378–387. doi:10.1007/978-3-319-07221-0_47.

[28] A. Mitrovic, Fifteen years of constraint-based tutors: what we have achieved and where we are going, User Modeling and User-Adapted Interaction 22 (1-2) (2011) 39–72. doi:10.1007/s11257-011-9105-9.

[29] K. Simons, P. Ertmer, Scaffolding disciplined inquiry in problem-based environments, International Journal of Learning 12 (11 2005).

[30] L. M. Razzaq, N. T. Heffernan, Scaffolding vs. hints in the assistment system, in: Intelligent Tutoring Systems, 8th International Conference, ITS 2006, Jhongli, Taiwan, June 26-30, 2006, Proceedings, Vol. 4053 of Lecture Notes in Computer Science, Springer, 2006, pp. 635–644. doi:10.1007/11774303_63.

[31] E. O'Rourke, E. Andersen, S. Gulwani, Z. Popović, A framework for automatically generating interactive instructional scaffolding, in: Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems, CHI '15, Association for Computing Machinery, New York, NY, USA, 2015, p. 1545–1554. doi:10.1145/2702123.2702580.

[32] O. Sychev, M. Gumerov, Generating pedagogical questions to help students learn, in: C. Frasson, P. Mylonas, C. Troussas (Eds.), Augmented Intelligence and Intelligent Tutoring Systems, Springer Nature Switzerland, Cham, 2023, pp. 195–208. doi:10.1007/978-3-031-32883-1_17.

[33] N. J. Kim, B. R. Belland, A. E. Walker, Effectiveness of computer-based scaffolding in the context of problem-based learning for stem education: Bayesian meta-analysis, Educational Psychology Review 30 (2) (2017) 397–429. doi:10.1007/s10648-017-9419-1.

[34] Z. Risha, J. Barria-Pineda, K. Akhuseyinoglu, P. Brusilovsky, Stepwise help and scaffolding for java code tracing problems with an interactive trace table, in: Proceedings of the 21st Koli Calling International Conference on Computing Education Research, Koli Calling '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–10. doi:10.1145/3488042.3490508.

[35] Y. Zhou, J. M. L. Andres-Bray, S. Hutt, K. Ostrow, R. S. Baker, A comparison of hints vs. scaffolding in a MOOC with adult learners, in: Artificial Intelligence in Education - 22nd International Conference, AIED 2021, Utrecht, The Netherlands, June 14-18, 2021, Proceedings, Part II, Vol. 12749 of Lecture Notes in Computer Science, Springer, 2021, pp. 427–432. doi:10.1007/978-3-030-78270-2_76.

[36] D. Nicol, From monologue to dialogue: improving written feedback processes in mass higher education, Assessment & Evaluation in Higher Education 35 (5) (2010) 501–517. doi:10.1080/02602931003786559.

[37] D. von Winterfeldt, W. Edwards, Decision Analysis and Behavioral Research, Cambridge University Press, Cambridge, New York, 1986.

[38] B. Kamiński, M. Jakubczyk, P. Szufel, A framework for sensitivity analysis of decision trees, Central European Journal of Operations Research 26 (1) (2017) 135–159. doi:10.1007/s10100-017-0479-6.
URL https://doi.org/10.1007/s10100-017-0479-6

[39] D. Beckett, T. Berners-Lee, E. Prud'hommeaux, G. Carothers, Rdf 1.1 turtle, accessed: 2023-09-09 (2014).
URL https://www.w3.org/TR/turtle/

[40] Y. Kamennov, O. Sychev, Y. A. Orlova, Covering possible reasoning errors for intelligent tutoring systems: Order of expression evaluation case, in: Intelligent Tutoring Systems - 18th International Conference, ITS 2022, Bucharest, Romania, June 29 - July 1, 2022, Proceedings, 2022, pp. 65–74. doi:10.1007/978-3-031-09680-8_6.

[41] A. Dovier, C. Piazza, E. Pontelli, G. Rossi, Sets and constraint logic programming, ACM Trans. Program. Lang. Syst. 22 (5) (2000) 861–931. doi:10.1145/365151.365169.

[42] V. Aleven, J. Rowe, Y. Huang, A. Mitrovic, Handbook of Artificial Intelligence in Education, Edward Elgar Publishing Limited, Cheltenham, UK, 2023, Ch. Domain modeling for AIED systems with connections to modeling student knowledge: a review, pp. 127–169. doi:10.4337/9781800375413.00017.

[43] O. Sychev, A. Anikin, N. Penskoy, M. Denisov, A. Prokudin, CompPrehension - model-based intelligent tutoring system on comprehension level, Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 12677 LNCS (2021) 52–59. doi:10.1007/978-3-030-80421-3_6.

[44] J. J. Rotman, An introduction to the theory of groups, Springer-Verlag, New York; Berlin; Heidelberg [etc.], 1999.

[45] M.-C. de Marneffe, C. D. Manning, J. Nivre, D. Zeman, Universal Dependencies, Computational Linguistics 47 (2) (2021) 255–308. doi:10.1162/coli_a_00402.

[46] O. Sychev, N. Penskoy, G. Terekhov, Using software reasoning to determine domain-law violations and provide explanatory feedback: Expressions tutor example, in: Proceedings of the 14th International Conference on Computer Supported Education, CSEDU 2022, Online Streaming, April 22-24, 2022, Volume 1, 2022, pp. 116–123. doi:10.5220/0011070100003182.

[47] D. Norris, Short-term memory and long-term memory are still different., Psychological Bulletin 143 (9) (2017) 992–1009. doi:10.1037/bul0000108.
URL https://doi.org/10.1037/bul0000108

[48] N. Cowan, The magical number 4 in short-term memory: A reconsideration of mental storage capacity, Behavioral and Brain Sciences 24 (1) (2001) 87–114. doi:10.1017/s0140525x01003922.
URL https://doi.org/10.1017/s0140525x01003922

[49] M. Garnelo, M. Shanahan, Reconciling deep learning with symbolic artificial intelligence: representing objects and relations, Current Opinion in Behavioral Sciences 29 (2019) 17–23, artificial Intelligence. doi:https://doi.org/10.1016/j.cobeha.2018.12.010.
URL https://www.sciencedirect.com/science/article/pii/S2352154618301943

[50] D. B. Lenat, P. J. Durlach, Reinforcing math knowledge by immersing students in a simulated learning-by-teaching experience, International Journal of Artificial Intelligence in Education 24 (3) (2014) 216–250. doi:10.1007/s40593-014-0016-x.

[51] Jena - a free and open source Java framework for building Semantic Web and Linked Data applications, https://jena.apache.org, accessed: 2023-09-09.

[52] E. Pasternak, R. Fenichel, A. N. Marshall, Tips for creating a block language with blockly, in: 2017 IEEE Blocks and Beyond Workshop (B B), IEEE, 2017, pp. 21–24. doi:10.1109/BLOCKS.2017.8120404.

[53] O. Sychev, N. Penskoy, A. Prokudin, Generating expression evaluation learning problems from existing program code, in: International Conference on Advanced Learning Technologies, ICALT 2022, Bucharest, Romania, July 1-4, 2022, 2022, pp. 183–187. doi:10.1109/ICALT55010.2022.00061.